

Trabajo Práctico: Tax-as-Code

Materia: Teoría de Lenguajes y Autómatas (TLA)

Equipo G-124

Nombre	Apellido	Legajo	E-mail
Santiago	Cibeira	64560	scibeira@itba.edu.ar
Manuel	Suñol	64138	msunol@itba.edu.ar
Augusto	Ospal	63669	aospalmadeo@itba.edu.ar

1. Introducción

El presente proyecto tiene como objetivo el diseño e implementación de **Tax-as-Code**, un Lenguaje de Dominio Específico (DSL) enfocado en la declaración de normativas fiscales y reglas de facturación.

La principal problemática que aborda el proyecto es la fragilidad de los sistemas contables actuales (ERPs genéricos), donde la configuración de topes impositivos y alícuotas se realiza de forma manual en bases de datos o requiere la intervención directa del área de Sistemas. **Tax-as-Code** permite a los profesionales contables "legislar" las reglas mediante archivos de texto plano con una sintaxis natural y amigable, generando una fuente de verdad única, versionable e inmutable.

2. Arquitectura del Compilador

El sistema fue construido utilizando el pipeline clásico de compilación mediante las herramientas **Flex** y **Bison** sobre lenguaje **C**, estructurado en las siguientes fases:

2.1. Análisis Léxico (Flex)

Se definieron patrones regulares (`FlexPatterns.l`) para reconocer los tokens del lenguaje. Se abstraieron conceptos numéricos (`NUMBER`), cadenas de texto (`STRING`), identificadores de variables (`IDENTIFIER`) y palabras clave reservadas (`KW_REGLA_PAGO` , `KW_IGUAL_A` , `KW_MAYOR_O_IGUAL_A` , etc.). El lexer filtra automáticamente los espacios y comentarios.

2.2. Análisis Sintáctico (Bison)

Se estableció una gramática libre de contexto (`BisonGrammar.y`) que define la estructura jerárquica del lenguaje. La gramática soporta:

- **Declaraciones:** Definición de constantes (`definir TOPE: 1000`).
- **Reglas:** Estructuras que encapsulan un conjunto de condiciones (`regla_pago "Nombre" { ... }`).
- **Condiciones:** Expresiones relacionales (`monto mayor a 1000`) y soporte para listas de strings y booleanos.

Durante esta fase se resolvieron conflictos de tipo *Shift/Reduce* mediante directivas de precedencia para diferenciar operadores matemáticos de viñetas de lista.

2.3. Árbol Sintáctico Abstracto (AST)

El Parser de Bison no solo valida la sintaxis, sino que construye en memoria un Árbol Sintáctico Abstracto (AST). Cada nodo representa una abstracción semántica (Regla, Condición, Valor), permitiendo desacoplar el análisis de la generación de código.

2.4. Análisis Semántico

Un módulo especializado (`SemanticAnalyzer.c`) recorre el AST verificando la coherencia del código antes de compilarlo. Entre sus tareas se incluye:

- **Tabla de Símbolos:** Verifica que las variables utilizadas en las condiciones (ej. `monto` , `es_monotributista` , `concepto`) estén correctamente declaradas y formen parte del dominio del negocio.
- **Chequeo de Tipos:** Asegura que no se comparen variables booleanas con cadenas de texto o números.

2.5. Generación de Código (Backend)

En línea con las especificaciones del proyecto (Stage 3), el compilador exporta la lógica de negocio a los formatos requeridos:

- **Archivos Planos (.txt) Normativos:** Archivo `afip_arba_configuration.txt` exportado en formato posicional de longitud fija para la potencial importación en aplicativos (SICORE, SIAp).
- **Planilla de Auditoría (.csv):** Archivo `audit_matrix.csv` con formato tabular para la conciliación de reglas y atributos por parte de los contadores, donde pueden filtrar y visualizar operaciones.
- **JSON Estructurado (Extra):** Un archivo `tax_rules.json` que desglosa matemáticamente las reglas y condiciones, listo para ser consumido por cualquier sistema externo.

3. Casos de Uso y Aplicación

Para validar la utilidad del DSL, se implementaron reglas que atacan problemas comunes en la contabilidad argentina:

Caso A: Prevención de Evasión de Monotributistas

```
definir TOPE_FACTURACION: 1800000
regla_facturacion "Alerta de Facturacion Apocrifia" {
  - es_monotributista igual a verdadero
  - monto mayor o igual a TOPE_FACTURACION
  - cantidad_facturas_consecutivas mayor o igual a 3
}
```

Caso B: Retención de Ganancias por Alquileres a Empresas

```
regla_pago "Retencion Ganancias - Alquiler" {
  - es_monotributista igual a falso
  - concepto igual a "alquileres"
  - monto mayor o igual a 1000000
}
```

4. Integración Externa: El Motor Transaccional (Mini ERP)

Para demostrar la versatilidad del archivo JSON generado, se construyó un ecosistema satélite: un **Mini ERP desarrollado en Golang**.

Este sistema está completamente **contenedorizado utilizando Docker** y corre de forma independiente del compilador. Se integra en el mismo archivo `compose.yaml` bajo el servicio `mini-erp`, exponiendo la interfaz en el puerto `8080`. Su función es recibir facturas entrantes, cargar en vivo el archivo `tax_rules.json` (gracias a un volumen compartido de Docker) y evaluar las condiciones fiscalmente.

El sistema liquida impuestos bajo dos esquemas independientes y complementarios:

1. **Retención de Ganancias:** Evaluación dinámica mediante el compilador de las condiciones declaradas en el archivo `.tac` (eximiendo monotributistas y aplicando alícuotas del 6.00% sobre conceptos como alquileres).
2. **Retención de Ingresos Brutos (IIBB):** Consulta en tiempo real de un padrón tributario simulado que discrimina por jurisdicción (ARBA para Provincia de Buenos Aires y AGIP para CABA), asignando alícuotas según el CUIT y la región (con tasas del 0% para exentos, del 1.5% al 3% para activos, y tasas penales del 4.0% para ARBA y 4.5% para AGIP en CUITs no empadronados).

Al procesar la factura, el sistema calcula de forma diferenciada ambas retenciones, liquida el importe neto a pagar y emite certificados oficiales separados y transparentes para cada tributo.

5. Conclusión

La separación de responsabilidades demostró ser altamente efectiva. Mediante **Tax-as-Code**, la lógica impositiva volátil queda en manos del sector contable bajo estricto control de versiones (Git), mientras que los sistemas de gestión centralizados se limitan a leer un JSON estándar, garantizando una arquitectura robusta, auditable y libre de fallas críticas en los despliegues.

6. Referencias

(No se han realizado referencias explícitas dentro del texto del documento).

7. Bibliografía

- Levine, J. R. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.
- GNU Project. (n.d.). *Bison - The Yacc-compatible Parser Generator*. Free Software Foundation.
- GNU Project. (n.d.). *Flex - The Fast Lexical Analyzer*. Free Software Foundation.